UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 20

Taller 3 — Calculadora financiera

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Objetivos de la sesión **2**

2 Prerrequisitos **2**

3 Arranque: preguntas de sondeo **2**

4 Solución: módulo `matematicas` **3**

5 Solución: módulo `financiero` **3**

6 Solución: módulo `interfaz` **5**

7 Solución: `main.cpp` **8**

8 Makefile de referencia **8**

9 Errores frecuentes **9**

10 Rúbrica de evaluación **10**

11 Conexiones con temas posteriores **10**

12 Materiales **11**

Objetivos de la sesión

Al finalizar el taller el alumno habrá:

1. Implementado un proyecto de consola en C++ distribuido en cuatro módulos (`matematicas`, `financiero`, `interfaz`, `main`).
2. Practicado la separación de responsabilidades: cálculo vs. interfaz.
3. Escrito un `Makefile` con variables que compile por etapas (`.cpp` → `.o` → ejecutable).
4. Verificado su implementación contra la tabla de pruebas de la especificación.

90 minutos. Distribución sugerida:

5 min	presentar el taller, resolver dudas de la especificación
10 min	repaso rápido de estructura de archivos y Makefile
60 min	trabajo individual (o por parejas, a criterio del profesor)
15 min	verificación con tabla de pruebas y cierre

A diferencia de las sesiones de teoría, aquí el profesor circula y atiende dudas. No hay exposición frontal sostenida. Esta sesión **no tiene cuaderno Jupyter**: el trabajo es exclusivamente en terminal y editor de texto.

Prerrequisitos

- Proyectos de múltiples archivos y Makefile (sesión 19).
- Parámetros de salida por referencia (sesión 15).
- Operaciones aritméticas básicas en C++ (sesión 2).

Si un alumno no recuerda cómo escribir el Makefile, remitirlo a las notas de la sesión 19 antes de guiarlo verbalmente. El Makefile de referencia está en la sección 8 de estas notas.

Arranque: preguntas de sondeo

1. "¿Para qué sirve un módulo que solo calcula, sin `cout` ni `cin`?"
Respuesta esperada: se puede reutilizar y probar de forma independiente; se puede cambiar la interfaz (terminal, GUI, web) sin tocar la lógica de negocio.

2. "¿Qué diferencia hay entre interés simple y compuesto?"
Respuesta esperada: en simple el interés siempre se calcula sobre el capital original; en compuesto el interés de cada período se acumula al capital y genera interés nuevo. Si nadie lo sabe, no explicar todavía: dejar que lo descubran en la

Parte II de sus notas.

3. "¿Cuántos `.cpp` va a tener tu proyecto?"
Respuesta esperada: cuatro — `main.cpp`, `matematicas.cpp`, `financiero.cpp` e `interfaz.cpp`. Si un alumno responde "uno", señalarle la sección 2 de las notas del alumno.

Solución: módulo `matematicas`

Las tres funciones son sencillas. El error más común es usar `pow()` de `<cmath>` en `potencia`; la especificación lo prohíbe explícitamente.

```
#include "matematicas.h"
#include <cmath>     // round()

double potencia(double base, int exp) {
    double r = 1.0;
    for (int i = 0; i < exp; i++)
        r *= base;
    return r;
}

double porcentaje(double valor, double pct) {
    return valor * (pct / 100.0);
}

double redondear(double val) {
    return round(val * 100.0) / 100.0;
}
```

- `potencia(b, 0)` devuelve `1.0` sin código especial: el ciclo no ejecuta ninguna iteración.
- `porcentaje(500, 16)` devuelve `80.0`. Error frecuente: olvidar dividir entre 100, obteniendo `8000.0`.
- `redondear` necesita `<cmath>`. Si el alumno omite el `#include`, el error señalará `matematicas.cpp`, no `financiero.cpp`.

Solución: módulo `financiero`

Este módulo incluye `matematicas.h` para usar `potencia` y `porcentaje`. La violación más grave que puede cometer un alumno aquí es agregar `cout` o `cin`.

```
#include "financiero.h"
```

```
#include "matematicas.h"

double interes_simple(double capital, double tasa, double tiempo) {
    return capital * (1.0 + (tasa / 100.0) * tiempo);
}

double interes_compuesto(double capital, double tasa, int periodos) {
    return capital * potencia(1.0 + tasa / 100.0, periodos);
}

double calcular_iva(double precio, double tasa) {
    return porcentaje(precio, tasa);
}

double desglosar_iva(double precio_con_iva, double tasa) {
    return precio_con_iva / (1.0 + tasa / 100.0);
}

double cuenta_restaurante(double subtotal, double pct_propina,
                          int adultos, int ninos,
                          double& total, double& pago_n) {
    double propina = porcentaje(subtotal, pct_propina);
    total          = subtotal + propina;
    double pago_a  = total / (adultos + ninos / 2.0);
    pago_n         = pago_a / 2.0;
    return pago_a;
}

double descuento(double precio, double pct_descuento) {
    return precio - porcentaje(precio, pct_descuento);
}

double convertir_divisas(double monto, double tipo_cambio) {
    return monto * tipo_cambio;
}
```

En `cuenta_restaurante` el divisor es $n_a + n_n/2$. Si el alumno escribe `ninos / 2` y `ninos` es `int`, la división es entera: con 1 niño, `1 / 2 = 0`. El niño no cuenta para dividir la cuenta.

La prueba de la tabla (2 adultos, 1 niño) expone el bug de inmediato: el pago por adulto sale $276.00 en lugar de $220.80.

Corrección: `ninos / 2.0` fuerza la división real.

"¿La tasa entra como 5 o como 0.05?"
Como `5`: el usuario teclea 5 para 5 %. La conversión a decimal se hace dentro de la función (`tasa / 100.0`).

"¿F3 y F4 deberían dar el mismo número si uso la salida de una como entrada de la otra?"
Sí. `calcular_iva(500, 16) = 80` → total = 580. `desglosar_iva(580, 16) = 500`. Si no cuadra, el error suele estar en F4: confundir `precio / tasa` con `precio / (1 + tasa)`.

"¿`interes_simple` retorna el interés o el monto?"
El **monto final** ($M = C + I$). El llamador en `interfaz` calcula $I = M - C$ para mostrarlo al usuario.

"¿Necesito incluir `matematicas.h` en `financiero.h`?"
No. `financiero.h` solo declara las firmas de las funciones financieras; no llama a nada de `matematicas`. La dependencia está en `financiero.cpp`, que es el que implementa las llamadas.

Solución: módulo `interfaz`

Las funciones de interfaz necesitan `<iomanip>` para `setprecision`. El estilo de presentación (columnas, etiquetas) queda a criterio del alumno; lo importante es que los números salgan con dos decimales y que la opción seleccionada sea visible en la captura.

```
#include "interfaz.h"
#include "financiero.h"
#include <iostream>
#include <iomanip>
using namespace std;

int mostrar_menu() {
    cout << "\n=== Calculadora Financiera ===\n"
         << " 1. Interes simple\n"
         << " 2. Interes compuesto\n"
         << " 3. Calcular IVA\n"
         << " 4. Desglosar IVA\n"
         << " 5. Cuenta de restaurante\n"
         << " 6. Descuento comercial\n"
         << " 7. Conversion de divisas\n"
         << " 0. Salir\n"
         << "Opcion: ";
    int op;
```

```
    cin >> op;
    return op;
}

void ejecutar_interes_simple() {
    double capital, tasa, tiempo;
    cout << "Capital inicial:     "; cin >> capital;
    cout << "Tasa de interes (%): "; cin >> tasa;
    cout << "Periodos:            "; cin >> tiempo;
    double monto = interes_simple(capital, tasa, tiempo);
    cout << fixed << setprecision(2)
         << "Interes generado: $" << (monto - capital) << "\n"
         << "Monto final:      $" << monto << "\n";
}

void ejecutar_interes_compuesto() {
    double capital, tasa;
    int    periodos;
    cout << "Capital inicial:     "; cin >> capital;
    cout << "Tasa de interes (%): "; cin >> tasa;
    cout << "Periodos:            "; cin >> periodos;
    double monto = interes_compuesto(capital, tasa, periodos);
    cout << fixed << setprecision(2)
         << "Ganancia:    $" << (monto - capital) << "\n"
         << "Monto final: $" << monto              << "\n";
}

void ejecutar_iva() {
    double precio, tasa;
    cout << "Precio sin IVA:  "; cin >> precio;
    cout << "Tasa de IVA (%): "; cin >> tasa;
    double iva = calcular_iva(precio, tasa);
    cout << fixed << setprecision(2)
         << "IVA:   $" << iva             << "\n"
         << "Total: $" << (precio + iva)  << "\n";
}

void ejecutar_desglosar_iva() {
    double precio_total, tasa;
    cout << "Precio con IVA:  "; cin >> precio_total;
    cout << "Tasa de IVA (%): "; cin >> tasa;
    double base = desglosar_iva(precio_total, tasa);
    cout << fixed << setprecision(2)
         << "Precio base: $" << base << "\n"
```

```
63          << "IVA:          $" << (precio_total - base)
               << "\n";
64  }
65
66  void ejecutar_restaurante() {
67      double subtotal, pct_propina;
68      int    adultos, ninos;
69      cout << "Subtotal de la cuenta: "; cin >> subtotal;
70      cout << "Propina (%):           "; cin >> pct_propina;
71      cout << "Adultos:               "; cin >> adultos;
72      cout << "Ninos:                 "; cin >> ninos;
73      double total, pago_n;
74      double pago_a = cuenta_restaurante(subtotal,
           pct_propina,
75                                         adultos, ninos,
76                                         total, pago_n);
77      cout << fixed << setprecision(2)
78           << "Propina:     $" << (total - subtotal) << "\n"
79           << "Total:       $" << total               <<
               "\n"
80           << "Pago adulto: $" << pago_a              <<
               "\n"
81           << "Pago nino:   $" << pago_n              <<
               "\n";
82  }
83
84  void ejecutar_descuento() {
85      double precio, pct;
86      cout << "Precio original: "; cin >> precio;
87      cout << "Descuento (%):   "; cin >> pct;
88      double precio_f = descuento(precio, pct);
89      cout << fixed << setprecision(2)
90           << "Ahorro:       $" << (precio - precio_f) <<
               "\n"
91           << "Precio final: $" << precio_f             <<
               "\n";
92  }
93
94  void ejecutar_divisas() {
95      double monto, tc;
96      cout << "Monto a convertir: "; cin >> monto;
97      cout << "Tipo de cambio:    "; cin >> tc;
98      cout << fixed << setprecision(2)
99           << "Equivalente: " << convertir_divisas(monto,
               tc) << "\n";
100 }
```

- `fixed « setprecision(2)` se aplica una sola vez y afecta todos los `cout` posteriores. Los alumnos a veces se sorprenden de que el segundo número

también salga con dos decimales — es el comportamiento esperado.

- El cálculo `monto - capital` ocurre en `interfaz.cpp`, no en `financiero.cpp`. Esto es correcto: `financiero` retorna el monto final e `interfaz` deriva el interés para el usuario.
- Las etiquetas y el formato visual (columnas, símbolos de pesos) quedan a criterio del alumno. Lo que se evalúa es que los valores numéricos sean correctos con dos decimales.

Solución: `main.cpp`

`main.cpp` solo incluye `interfaz.h`. No toca `financiero.h` ni `matematicas.h` directamente.

```cpp
#include "interfaz.h"
#include <iostream>
using namespace std;

int main() {
    int opcion;
    do {
        opcion = mostrar_menu();
        switch (opcion) {
            case 1: ejecutar_interes_simple();     break;
            case 2: ejecutar_interes_compuesto(); break;
            case 3: ejecutar_iva();                break;
            case 4: ejecutar_desglosar_iva();      break;
            case 5: ejecutar_restaurante();        break;
            case 6: ejecutar_descuento();          break;
            case 7: ejecutar_divisas();            break;
            case 0: cout << "Hasta luego.\n";      break;
            default: cout << "Opcion no valida.\n"; break;
        }
    } while (opcion != 0);
    return 0;
}
```

Makefile de referencia

```makefile
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra
TARGET   = calculadora
SRCS     = main.cpp matematicas.cpp financiero.cpp interfaz.cpp
```

```
OBJS     = $(SRCS:.cpp=.o)

$(TARGET): $(OBJS)
 $(CXX) $(OBJS) -o $(TARGET)

%.o: %.cpp
 $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
 rm -f $(OBJS) $(TARGET)
```

La sustitución `$(SRCS:.cpp=.o)` genera `OBJS` automáticamente. Si un alumno prefiere listar los `.o` explícitamente, también es válido mientras compile en dos etapas.

Errores frecuentes

1. `cout/cin` en `financiero.cpp`.
Síntoma: el programa funciona, pero la separación está rota.
Cómo detectarlo: preguntar "¿dónde usas `cout`?". Si señala `financiero.cpp`, pedirle que mueva todo el I/O a `interfaz.cpp`.

2. División entera en F5.
Síntoma: F5 con 2 adultos y 1 niño da pago adulto $276.00 en lugar de $220.80.
Causa: `ninos / 2` (entero) en lugar de `ninos / 2.0`.

3. Tasa sin dividir entre 100.
Síntoma: F1 con capital 1000, tasa 5, tiempo 3 da 16 000 en lugar de 1 150.
Causa: `capital * tasa * tiempo` en lugar de `capital * (tasa / 100.0) * tiempo`.

4. Fórmula de F4 incorrecta.
Síntoma: F4(580, 16) da ≈497.41 en lugar de 500.00.
Causa: `precio / tasa` en lugar de `precio / (1.0 + tasa / 100.0)`.
Verificación rápida: F3 y F4 deben ser inversas entre sí.

5. Olvidar `fixed « setprecision(2)`.
Síntoma: resultados con muchos decimales o en notación científica.
Corrección: `cout « fixed « setprecision(2) « valor`. Recordar incluir `<iomanip>`.

6. Tab sustituido por espacios en el Makefile.
Error: `Makefile:N: *** missing separator. Stop.`
Remedio: el comando de cada regla empieza con Tab. En VS Code, verificar que el modo de indentación del archivo sea Tabs.

7. `potencia` implementada con `pow()`.
La especificación prohíbe usar `pow()` en `potencia`. Pedirle al alumno que la implemente con un ciclo.

Rúbrica de evaluación

La entrega es un PDF con capturas de pantalla de la terminal. El código fuente no se entrega.

Criterio	Pts.	Evidencia
Makefile (captura de `cat Makefile`)		
Variables `CXX`, `CXXFLAGS`, `TARGET`, `SRCS`	8	presentes y usadas
Compilación por etapas (`.cpp` → `.o` → ejecutable)	7	regla patrón `%.o` visible
Compilación limpia		
`make clean` + `make` sin errores ni advertencias	10	captura de terminal
Funcionalidades (7 funciones × 8 pts)		
F1 — Interés simple	8	opción, datos y resultado visibles
F2 — Interés compuesto	8	ídem
F3 — Calcular IVA	8	ídem
F4 — Desglosar IVA	8	ídem
F5 — Cuenta restaurante	8	ídem
F6 — Descuento comercial	8	ídem
F7 — Conversión divisas	8	ídem
Casos adicionales		
≥ 3 funciones con 2 capturas distintas c/u	9	datos diferentes a los de la tabla de pruebas
Total	**100**	

- Advertencias de `-Wall -Wextra` en la captura de compilación: -3 pts de la sección "compilación limpia".
- F3 y F4 inconsistentes entre sí (F3 $\circ$ F4 $\neq$ identidad): -8 pts de F4.
- Makefile sin regla `clean`: -3 pts.

Conexiones con temas posteriores

- **Orientación a objetos** — El módulo `financiero` es un candidato natural para convertirse en una clase `Calculadora`: los métodos serían exactamente estas funciones. El alumno ya escribió la interfaz pública de la clase sin saberlo.
- **Pruebas unitarias** — Un módulo que solo calcula, sin I/O, es trivialmente testeable. La separación practicada hoy es el prerequisito para poder escribir pruebas automatizadas.
- **Librerías** — Si `matematicas.o` y `financiero.o` se empaquetaran en un `.a`, cualquier otro programa podría usar las funciones sin recompilarlas. El mismo patrón siguen todas las librerías de C++.

Materiales

- **Notas alumno:** `notas_alumno/pdf/sesion20_taller_calculadora_financiera.pdf`
- Esta sesión no tiene cuaderno Jupyter.